

MODULE

Merge Sort

Merge sort is a divide and conquer sorting algorithm. It divides the array into smaller sub arrays until each sub array contains one element. Then, these sub arrays are merged in sorted order to obtain the final sorted list.

Time Complexity-

- Worst case: $O(n \log n)$
- Best case: $O(n \log n)$

Space Complexity

- $O(n)$

Advantages-

- Efficient for large datasets
- Stable sorting algorithm
- Uses divide and conquer technique

Date: _____

Problem 1- Rank student by Marks

```
#include <stdio.h>

void merge(int arr[], int low,
           int mid, int high)
{
    int i = low;
    int j = mid + 1;
    int k = low;
    int temp[100];
    while(i <= mid && j <= high) {
        if(arr[i] < arr[j]) {
            temp[k] = arr[i];
            i++;
        }
        else {
            temp[k] = arr[j];
            j++;
        }
        k++;
    }
    while(i <= mid) {
        temp[k] = arr[i];
        j++;
        k++;
    }
    for(i = low; i <= high; i++) {
```

```
arr[i] = temp[i];
}
```

```
void merge sort(int arr[], int low,
                int high)
```

```
{
```

```
    int mid;
```

```
    if (low < high) {
```

```
        mid = (low + high) / 2;
```

```
        merge sort(arr, low, mid);
```

```
        merge sort(arr, mid+1, high);
```

```
        merge (arr, low, mid, high);
```

```
    }
```

```
int main () {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int arr[100];
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    merge sort(arr, 0, n-1);
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        printf("%d", arr[i]);
```

```
    }
```

```
    return 0;
```

```
}
```


RUN AND DEBUG

VARIABLES

WATCH

CALL STACK

BREAKPOINTS

Welcome

program.c

C/C++: clang build and ...

```
Users > macbook > hello.c > C program.c > merge(int [], int, int, int)
1  #include <stdio.h>
2  void merge(int arr[], int low, int mid, int high){
3      int i = low;
4      int j = mid + 1;
5      int k = low;
6      int temp[100];
7      while (i <= mid && j <= high) {
8          if (arr[i] < arr[j]){
9              temp[k] = arr[i];
10             i++;
11         }
12         else{
13             temp[k] = arr[j];
14             j++;
15         }
16         k++;
17     }
18     while (i <= mid){
19         temp[k] = arr[i];
20         i++;
21         k++;
22     }
23     while (j <= high){
24         temp[k] = arr[j];
25         j++;
26         k++;
27     }
28     for (i = low; i <= high; i++){
29         arr[i] = temp[i];
30     }
31 }
32 void mergeSort(int arr[], int low, int high){
33     int mid;
34     if (low < high){
35         mid = (low + high) / 2;
36         mergeSort(arr, low, mid);
37         mergeSort(arr, mid + 1, high);
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

macbook@macbooks-MacBook-Pro hello.c % ./Users/macbook/hello.c/program

6

67 56 45 87 89 57

45 56 57 67 87 89

macbook@macbooks-MacBook-Pro hello.c %

Running

Ln 2, Col 51 Spaces: 4 UTF-8 LF C Mac

RUN AND DEBUG

VARIABLES

WATCH

CALL STACK

BREAKPOINTS

Welcome

program.c

Users > macbook > hello.c > C: program.c > merge(int [], int, int, int)

```
2 void merge(int arr[], int low, int mid, int high){
28     for (i = low; i <= high; i++) {
29         arr[i] = temp[i];
30     }
31 }
32 void mergeSort(int arr[], int low, int high){
33     int mid;
34     if (low < high){
35         mid = (low + high) / 2;
36         mergeSort(arr, low, mid);
37         mergeSort(arr, mid + 1, high);
38         merge(arr, low, mid, high);
39     }
40 }
41 int main(){
42     int n;
43     scanf("%d", &n);
44     int arr[100];
45     for (int i = 0; i < n; i++) {
46         scanf("%d", &arr[i]);
47     }
48     mergeSort(arr, 0, n - 1);
49     for (int i = 0; i < n; i++){
50         printf("%d ", arr[i]);
51     }
52     return 0;
53 }
54
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

macbook@macbooks-MacBook-Pro hello.c % /Users/macbook/hello.c/program

6

67 56 45 87 89 57

45 56 57 67 87 89

macbook@macbooks-MacBook-Pro hello.c %

gcc build...

/Users/mac...

Ln 2, Col 51 Spaces: 4 UTF-8 LF C Mac

Problem 2- Organizing Product Prices

```
#include <stdio.h>

void merge (int price[], int mid,
            int start, int end)
{
    int a = start;
    int b = mid + 1;
    int c = start;
    int temp[100];
    while (a <= mid && b <= end)
    {
        if (price[a] < price[b]) {
            temp[c] = price[a];
            a++;
        }
        else {
            temp[c] = price[b];
            b++;
        }
        c++;
    }
    while (a <= end) {
        temp[c] = price[a];
        a++;
        c++;
    }
    while (b <= end) {
```



```
temp[c] = price[b];  
    b++;  
    c++;
```

```
}
```

```
for(a = start; a <= end; a++) {  
    price[a] = temp[a];  
}
```

```
void merge sort (int price[],  
                 int start, int end)  
{
```

```
    int mid;
```

```
    if (start < end)
```

```
{
```

```
        mid = (start + end) / 2;
```

```
        merge sort (price, start, mid);
```

```
        merge sort (price, mid + 1, end);
```

```
        merge (price, start, mid, end);  
    }
```

```
int main () {
```

```
    int n;
```

```
    scanf ("%d", &n);
```

```
    int price[100];
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf ("%d", &price[i]);
```

```
    }
```

```
    merge sort (price, 0, n - 1);
```

```
    printf ("Sorted prices: %d", n);
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf ("%d", price[i]);
```

3

section 0;

3

RUN AND DEBUG

VARIABLES

WATCH

Users > macbook > hello.c > C program.c > main()

```
1 #include <stdio.h>
2 void merge(int price[], int start, int mid, int end)
3 {
4     int a = start;
5     int b = mid + 1;
6     int c = start;
7     int temp[100];
8     while (a <= mid && b <= end)
9     {
10         if (price[a] < price[b])
11         {
12             temp[c] = price[a];
13             a++;
14         }
15         else
16         {
17             temp[c] = price[b];
18             b++;
19         }
20         c++;
21     }
22     while (a <= mid)
23     {
24         temp[c] = price[a];
25         a++;
26         c++;
27     }
28     while (b <= end)
29     {
30         temp[c] = price[b];
31         b++;
32         c++;
33     }
34     for (a = start; a <= end; a++)
35     {
36         price[a] = temp[a];
37     }
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
macbook@macbooks-MacBook-Pro hello.c % ./Users/macbook/hello.c/program
6
78 667 79 575 689
787
Sorted Prices:
78 79 575 667 689 787
macbook@macbooks-MacBook-Pro hello.c %
```

CALL STACK

BREAKPOINTS

Running

Ln 65, Col 6 Spaces: 4 UTF-8 LF C Mac

RUN AND DEBUG

VARIABLES

WATCH

CALL STACK

BREAKPOINTS

Welcome

C program.c X

Users > macbook > hello.c > C program.c > main()

```
1 void merge(int price[], int start, int mid, int end)
2   for (a = start; a <= end; a++)
3   {
4       price[a] = temp[a];
5   }
6 }
7
8 void mergeSort(int price[], int start, int end)
9 {
10     int mid;
11     if (start < end)
12     {
13         mid = (start + end) / 2;
14
15         mergeSort(price, start, mid);
16         mergeSort(price, mid + 1, end);
17         merge(price, start, mid, end);
18     }
19 }
20
21 int main()
22 {
23     int n;
24     scanf("%d", &n);
25     int price[100];
26     for (int i = 0; i < n; i++)
27     {
28         scanf("%d", &price[i]);
29     }
30     mergeSort(price, 0, n - 1);
31     printf("Sorted Prices:\n");
32     for (int i = 0; i < n; i++)
33     {
34         printf("%d ", price[i]);
35     }
36     return 0;
37 }
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

macbook@macbooks-MacBook-Pro hello.c % ./Users/macbook/hello.c/program
6
78 667 79 575 689
787
Sorted Prices:
78 79 575 667 689 787
macbook@macbooks-MacBook-Pro hello.c %

Ln 65, Col 6 Spaces: 4 UTF-8 LF C Mac

Problem 3- Sort employee records
by salary

```
#include <stdio.h>

void merge( int arr[], int low,
            int mid, int high)
{
    int i = low;
    int j = mid + 1;
    int k = low;
    int temp[100];
    while( i <= mid && j <= high)
    {
        if( arr[i] < arr[j] ) {
            temp[k] = arr[i];
            i++;
        }
        else {
            temp[k] = arr[j];
            j++;
            k++;
        }
    }
    while( i <= mid ) {
        temp[k] = arr[i];
        i++;
        k++;
    }
}
```



```

while(j <= high) {
    temp[k] = arr[j];
    j++;
    k++;
}

```

```

for(i = low; i <= high; i++) {
    arr[i] = temp[i];
}
}

```

```

void mergesort(int arr[], int low,
               int high) {

```

```

    int mid;
    if(low < high) {
        mid = (low + high) / 2;
        mergesort(arr, low, mid);
        mergesort(arr, mid + 1, high);
        merge(arr, low, mid, high);
    }
}

```

```

int main() {

```

```

    int n, diff;
    scanf("%i", &n);
    int arr[100];
    for(int i = 0; i < n; i++) {
        scanf("%i", &arr[i]);
    }

```

```

    mergesort(arr, 0, n - 1);
    for(int i = 0; i < n; i++) {
        printf("%i", arr[i]);
    }

```

```

    printf("%i", n);
}

```



```
printf("lowest salary:
```

```
%.1d\n", arr[0]);
```

```
printf("highest salary:
```

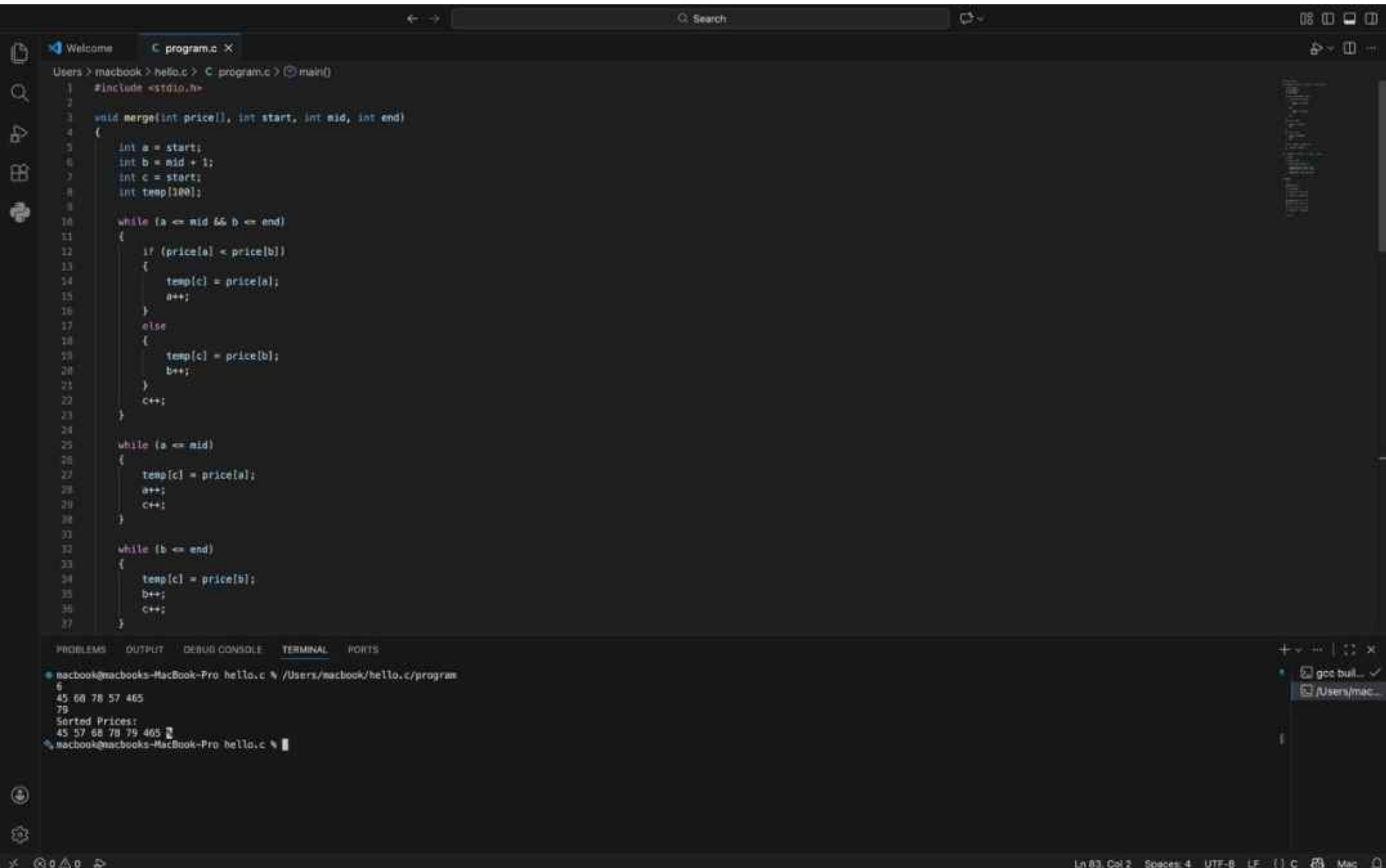
```
%.1d\n", arr[n-1]);
```

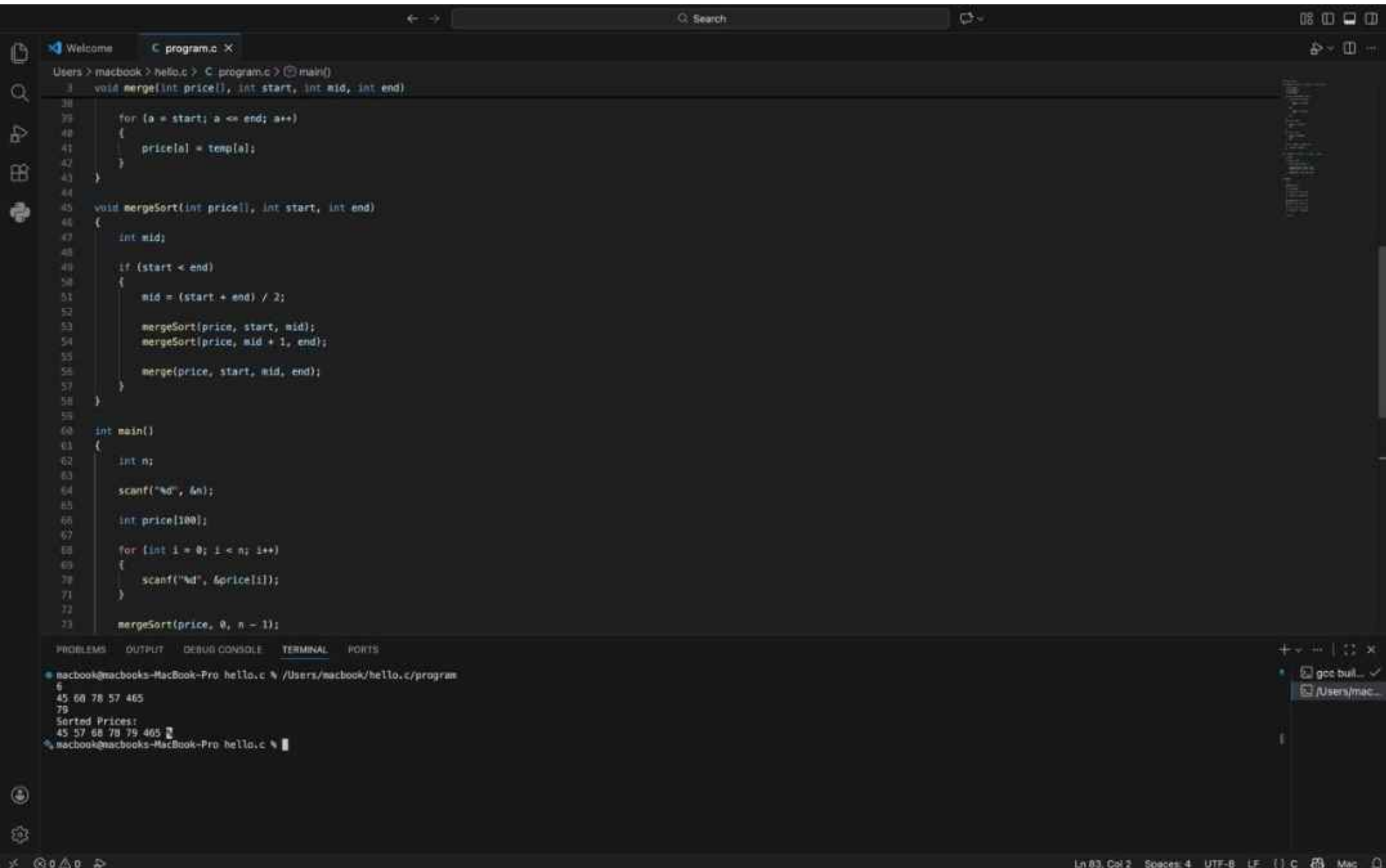
```
diff = arr[n-1] - arr[0];
```

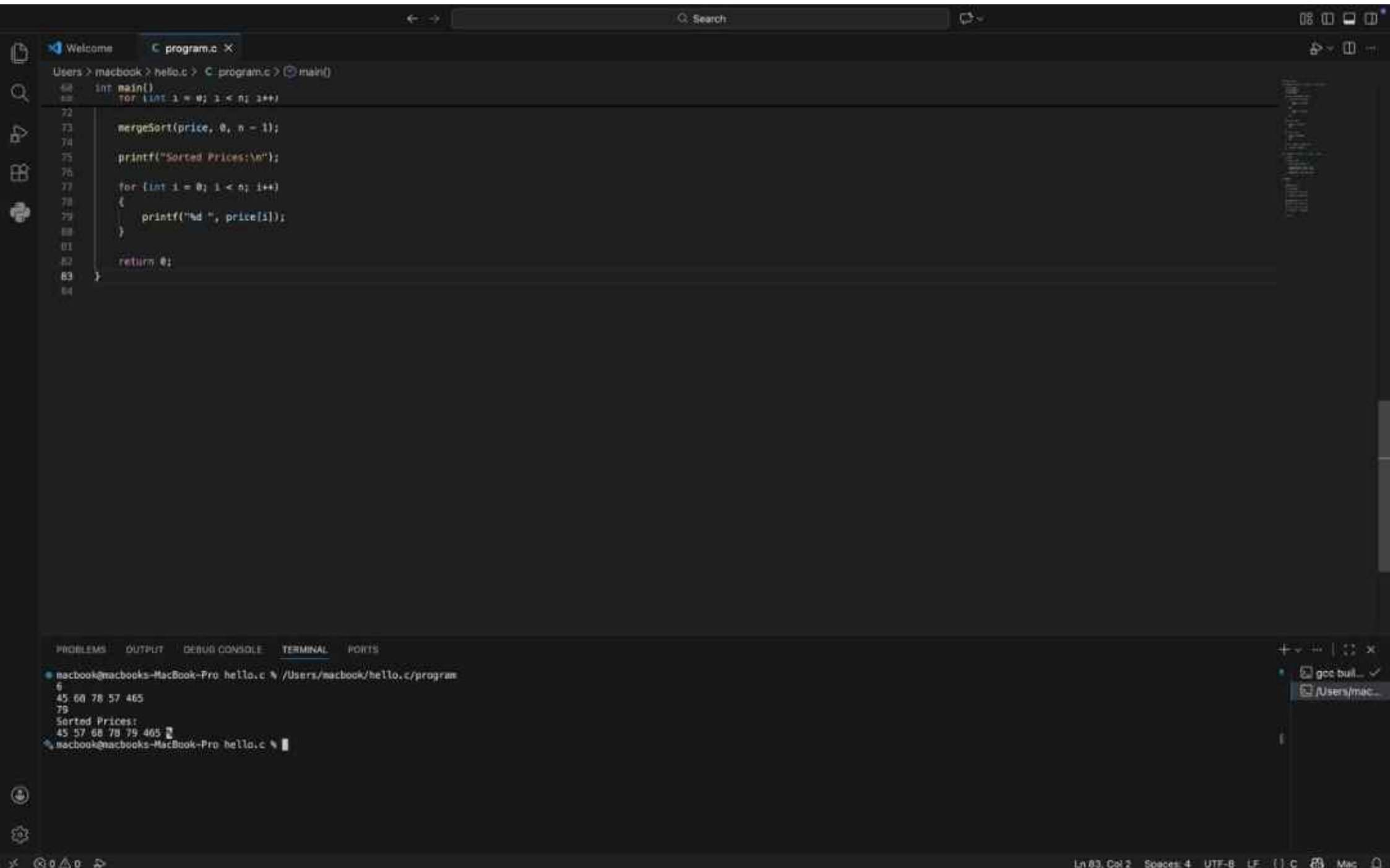
```
printf("difference: %.1d", diff);
```

```
return 0;
```

```
}
```







Quick Sort

Quick sort is a divide and conquer sorting algorithm. It selects a pivot element and places smaller elements on one side and larger element on the other side. The same process is repeated recursively until the entire array is sorted.

Time complexity

- Best case: $O(n \log n)$
- Worst case: $O(n^2)$

Space complexity

- $O(\log n)$

Advantages-

fast for large datasets

Uses less memory

Efficient in practice

Problem 1- Race Timing Analysis

```
#include <stdio.h>
void quick (int arr[], int low,
            int high)
{
```

```
    int p, x, y, temp;
```

```
    if (low < high) {
```

```
        p = arr[low];
```

```
        x = low;
```

```
        y = high;
```

```
        while (x < y) {
```

```
            while (arr[x] <= p && x < high)
            {
```

```
                x++;
```

```
            }
```

```
            while (arr[y] > p) {
```

```
                y--;
```

```
            }
```

```
            if (x < y) {
```

```
                temp = arr[x];
```

```
                arr[x] = arr[y];
```

```
                arr[y] = temp;
```

```
            } }
```

```
        temp = arr[low];
```

```
        arr[low] = arr[y];
```

```
        arr[y] = temp;
```



```
quick(arr, low, y-1);  
quick(arr, y+1, high);  
}
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int arr[100];
```

```
    for(int i=0; i<n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    quick(arr, 0, n-1);
```

```
    for(int i=0; i<n; i++)
```

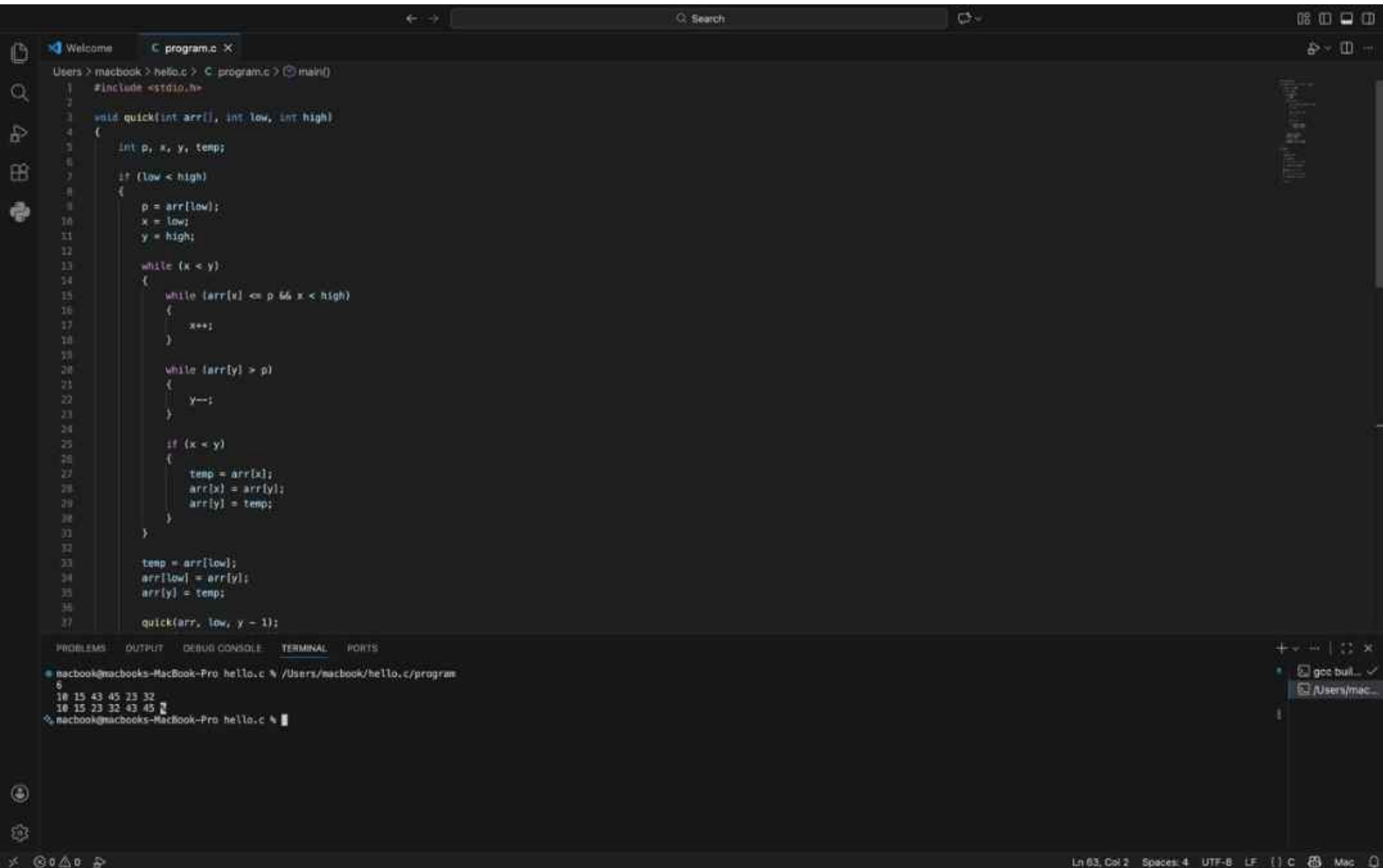
```
    {
```

```
        printf("%d", arr[i]);
```

```
    }
```

```
    return 0;
```

```
}
```

← → 🔍 Search

Welcome C program.c X

Users > macbook > hello.c > C:program.c > main()
3 void quick(int arr[], int low, int high)
7 { if (low < high)
37 { quick(arr, low, y - 1);
38 quick(arr, y + 1, high);
39 }
40 }
41
42 int main()
43 {
44 int n;
45
46 scanf("%d", &n);
47
48 int arr[100];
49
50 for (int i = 0; i < n; i++)
51 {
52 scanf("%d", &arr[i]);
53 }
54
55 quick(arr, 0, n - 1);
56
57 for (int i = 0; i < n; i++)
58 {
59 printf("%d ", arr[i]);
60 }
61
62 return 0;
63 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
• macbook@macbooks-MacBook-Pro hello.c % ./Users/macbook/hello.c/program
8
10 15 43 45 23 32
10 15 23 32 43 45
• macbook@macbooks-MacBook-Pro hello.c %

gcc build... ✓
/Users/mac...

Ln 63, Col 2 Spaces: 4 UTF-8 LF {} C Mac

apsara
Date: _____
Problem 2 - Organizing library Book
IDS

```
#include <stdio.h>
void sortId(int id[], int end,
            int start)
{
    int p, a, b, temp;
    if (start < end) {
        p = id[start];
        a = start;
        b = end;
        while (a < b) {
            while (id[a] <= p && a < end)
                a++;
            while (id[b] > p)
                b--;
            if (a < b) {
                temp = id[a];
                id[a] = id[b];
                id[b] = temp;
            }
            temp = id[start];
            id[start] = id[b];
            id[b] = temp;
            sortId(id, start, b-1);
        }
    }
}
```

```
sortId(id, b+1, end);  
}
```

```
int main () {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int id[100];
```

```
    for (int i = 0; i < n; i++)  
    {
```

```
        scanf("%d", &id[i]);  
    }
```

```
    sortId(id, 0, n-1);
```

```
    printf("sorted IDs: ");
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("%d ", id[i]);  
    }
```

```
    return 0;
```

```
}
```

← →

Search

08 11 11

Welcome C program.c X

Users > macbook > hello.c > C:program.c > main()

```
1 #include <stdio.h>
2
3 void quick(int arr[], int low, int high)
4 {
5     int p, x, y, temp;
6
7     if (low < high)
8     {
9         p = arr[low];
10        x = low;
11        y = high;
12
13        while (x < y)
14        {
15            while (arr[x] <= p && x < high)
16            {
17                x++;
18            }
19
20            while (arr[y] > p)
21            {
22                y--;
23            }
24
25            if (x < y)
26            {
27                temp = arr[x];
28                arr[x] = arr[y];
29                arr[y] = temp;
30            }
31        }
32
33        temp = arr[low];
34        arr[low] = arr[y];
35        arr[y] = temp;
36
37        quick(arr, low, y - 1);
38    }
39 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
macbook@macbooks-MacBook-Pro hello.c % ./Users/macbook/hello.c/program
8
10 15 43 45 23 32
10 15 23 32 43 45
macbook@macbooks-MacBook-Pro hello.c %
```

goc build... ✓
/Users/mac...

Ln 63, Col 2 Spaces: 4 UTF-8 LF C Mac

← →

Search

08 11 11

WelcomeC program.c X

Users> macbook> hello.c> C:program.c> main()
void sortId(int id[], int start, int end)
33 }
34 int main()
35 {
36 int n;
37 scanf("%d", &n);
38 int id[100];
39 for (int i = 0; i < n; i++)
40 {
41 scanf("%d", &id[i]);
42 }
43 sortId(id, 0, n - 1);
44 printf("Sorted IDs:\n");
45 for (int i = 0; i < n; i++)
46 {
47 printf("%d ", id[i]);
48 }
49 return 0;
50 }

PROBLEMSOUTPUTDEBUG CONSOLETERMINALPORTS

macbook@macbooks-MacBook-Pro hello.c % ./Users/macbook/hello.c/program
6
67 77 57 46 97 87
Sorted IDs:
46 57 67 77 87 97
macbook@macbooks-MacBook-Pro hello.c %

gcc buil...
/Users/mac...

Ln 49, Col 5 Spaces: 4 UTF-8 LF {} C Mac

Problem 3 - Online store sales Ranking

```
#include <stdio.h>

void quicksale (int sale[],
                int low, int high)
{
    int p, x, y, temp;
    if (low < high)
    {
        p = sale[low];
        x = low;
        y = high;
        while (x < y) {
            while (sale[x] >= p && x < high) {
                x++;
            }
            while (sale[y] < p) {
                y--;
            }
            if (x < y) {
                temp = sale[x];
                sale[x] = sale[y];
                sale[y] = temp;
            }
            quicksale (sale, low, y-1);
            quicksale (sale, y+1, high);
        }
    }
}
```


Date: _____
`int main () {`

`int n, sum = 0;`

`scanf ("%d", &n);`

`int sale[100];`

`for (int i = 0; i < n; i++) {`

`scanf ("%d", &sale[i]);`
`}`

`quicksort (sale, 0, n-1);`

`printf ("Sorted salary: \n");`

`for (int i = 0; i < n; i++) {`

`printf ("%d", sale[i]);`
`}`

`printf ("\n");`

`printf ("Top 3: ");`

`for (int i = 0; i < 3; i++) {`

`printf ("%d", sale[i]);`

`sum = sum + sale[i];`

`}`

`printf ("\n Average: %d", sum/3);`

`return 0;`

`}`

← →

Search

08 00 00

Welcome C program.c X

Users > macbook > hello.c > C:program.c > main()

```
1 #include <stdio.h>
2 void quickSale(int sale[], int low, int high)
3 {
4     int p, x, y, temp;
5     if (low < high)
6     {
7         p = sale[low];
8         x = low;
9         y = high;
10        while (x < y)
11        {
12            while (sale[x] >= p && x < high)
13            {
14                x++;
15            }
16            while (sale[y] < p)
17            {
18                y--;
19            }
20            if (x < y)
21            {
22                temp = sale[x];
23                sale[x] = sale[y];
24                sale[y] = temp;
25            }
26        }
27        temp = sale[low];
28        sale[low] = sale[y];
29        sale[y] = temp;
30        quickSale(sale, low, y - 1);
31        quickSale(sale, y + 1, high);
32    }
33 }
34 int main()
35 {
36     int n, sum = 0;
37     scanf("%d", &n);
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

macbook@macbooks-MacBook-Pro hello.c % ./Users/macbook/hello.c/program

6

46 46 43 68 32 39

Sorted Sales:

68 46 46 43 39 32

Top 3: 68 46 46

Average: 53.2

macbook@macbooks-MacBook-Pro hello.c %

gcc buil...

/Users/mac...

Ln 56, Col 38 Spaces: 4 UTF-8 LF C Mac

← →

Search

08 00 00

Welcome C program.c X

Users > macbook > hello.c > C:program.c > main()

```
35 {
36     int n, sum = 0;
37     scanf("%d", &n);
38     int sale[100];
39     for (int i = 0; i < n; i++)
40     {
41         scanf("%d", &sale[i]);
42     }
43     quickSale(sale, 0, n - 1);
44     printf("Sorted Sales:\n");
45     for (int i = 0; i < n; i++)
46     {
47         printf("%d ", sale[i]);
48     }
49     printf("\n");
50     printf("Top 3: ");
51     for (int i = 0; i < 3; i++)
52     {
53         printf("%d ", sale[i]);
54         sum = sum + sale[i];
55     }
56     printf("\nAverage: %d", sum / 3);
57     return 0;
58 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

macbook@macbooks-MacBook-Pro hello.c % /Users/macbook/hello.c/program
6
46 46 43 68 32 39
Sorted Sales:
88 46 46 43 39 32
Top 3: 68 46 46
Average: 53
macbook@macbooks-MacBook-Pro hello.c %

gcc build...
/Users/mac...

Ln 56, Col 38 Spaces: 4 UTF-8 LF C Mac